Analysis of Algorithms & Time Complexity

Theme 1: Technical Slate & Cool Mint | Detailed Proof Explanations

Q1. Determine the time complexity:

```
for(int i=0;i<n;i++) { cout<<i; }
```

DETAILED EXPLANATION:
The loop initializes at 0 and increments sequentially by 1 up to n-1. The block inside runs exactly n times, with each operation inside taking constant time O(1). Thus, execution scales linearly with the input size.

Time Complexity: O(n)

Q2. Determine the time complexity:

```
for(int i=0;i<n;i++) {
    for(int j=0;j<n;j++) { cout<<"*"; }
}
```

DETAILED EXPLANATION:
A classic nested loop configuration. The outer sequence executes n times. For each distinct iteration step of the outer structure, the inner loop independently loops n times. Total work equals the product: $n \times n = n^2$.

Time Complexity: $O(n^2)$

Q3. Determine the time complexity:

```
for(int i=0;i<n;i++) {
    for(int j=0;j<i;j++) { cout<<"*"; }
}
```

DETAILED EXPLANATION:
The inner statement limit relies directly on the changing outer index i. The inner iterations form an arithmetic progression: $0 + 1 + 2 + \ldots + (n-1) = n(n-1)/2$. Asymptotically, the dominant higher order power is quadratic.

Time Complexity: $O(n^2)$

Q4. Determine the time complexity:

```
for(int i=1;i<n;i*=2) { cout<<i; }
```

DETAILED EXPLANATION:
The index variable doubles on every iteration loop pass (1, 2, 4, 8...). It takes logμn steps to reach or exceed n, scaling the computational execution path logarithmically.

Time Complexity: O(log n)

Q5. Determine the time complexity:

```
for(int i=n;i>1;i/=2) { cout<<i; }
```

DETAILED EXPLANATION:
The input value starts at n and is continually divided by 2 until it reaches 1. This represents the reverse direction of logarithmic growth, maintaining an identical complexity of O(log n).

Time Complexity: O(log n)

Q6. Determine the time complexity:

```
for(int i=1;i<=n;i++) {
    for(int j=1;j<=log(n);j++) { cout<<"*"; }
}
```

DETAILED EXPLANATION:
The outer loop runs a total of n linear steps. The inner loop boundaries run independently up to a log(n) limit. Combining these two product environments directly yields n log n work.

Time Complexity: O(n log n)

Q7. Determine the time complexity:

```
while(n>0) { n--; }
```

DETAILED EXPLANATION:
The variable decreases monotonically by a constant unit value of 1 per cycle step. The total number of iterations matches the size of n exactly.

Time Complexity: O(n)

Q8. Determine the time complexity:

```
while(n>1) { n/=2; }
```

DETAILED EXPLANATION:
The loop bisection property repeatedly cuts the parameter space n in half until it hits the terminal loop floor of 1, taking logarithmic time.

Time Complexity: O(log n)

Q9. Determine the time complexity:

```
for(int i=1;i<=n;i++) {}
for(int j=1;j<=n;j++) {}
```

DETAILED EXPLANATION:
Two non-nested sequential statements tracking linear limits. Total time cost is evaluated by addition: O(n) + O(n) = O(2n), which simplifies asymptotically to O(n).

Time Complexity: O(n)

Q10. Determine the time complexity:

```
for(int i=1;i<=n;i++) {}
for(int j=1;j<=n*n;j++) {}
```

DETAILED EXPLANATION:
Two consecutive independent processing sequences. The first structure requires linear O(n) time steps while the second requires quadratic $O(n^2)$ passes. Asymptotically, the highest polynomial component dominates the execution profile.

Time Complexity: $O(n^2)$

Q11. Determine the time complexity:

```
for(int i=1;i<=n;i++) {
    for(int j=1;j<=n*n;j++) {}
}
```

DETAILED EXPLANATION:
Nested execution loop systems where the outer framework loops n times, and the embedded structure loops n^2 times per pass. Total algorithmic computation costs equal $n \times n^2 = n&sub3;$.

Time Complexity: $O(n^3)$

Q12. Determine the time complexity:

```
for(int i=1;i<=n;i*=3) {}
```

DETAILED EXPLANATION:
The loop index multiplies by a factor of 3 during each step sequence. This geometric progression resolves to a base-3 logarithmic complexity profile.

Time Complexity: O(log n)

Q13. Determine the time complexity:

```
for(int i=1;i<=n;i++) {
    for(int j=1;j<=i*i;j++) {}
}
```

DETAILED EXPLANATION:
The inner statement limit relies quadratically on the current outer iteration step index i. Summing up the quadratic iteration intervals yields a sum series dominated by the n^3 factor.

Time Complexity: O(n^3)

Q14. Determine the time complexity:

```
void fun(int n) {
    if(n==0) return;
    fun(n-1);
}
```

DETAILED EXPLANATION:
This recurrence tracks linearly via the equation T(n) = T(n-1) + O(1). The stack depth scales unit-by-unit downwards, creating a direct linear call string.

Time Complexity: O(n)

Q15. Determine the time complexity:

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
}
```

DETAILED EXPLANATION:
The recursion context divides the problem argument workspace by 2 at each sequential function call stack depth level, mirroring logarithmic structural behavior.

Time Complexity: O(log n)

Q16. Determine the time complexity:

```
void fun(int n) {
    if(n==0) return;
    fun(n-1);
    fun(n-1);
}
```

DETAILED EXPLANATION:
Every single tracking state forks into two separate linear paths, setting up a binary tree recurrence T(n) = 2T(n-1) + O(1), expanding exponentially to 2^n.

Time Complexity: $O(2^n)$

Q17. Determine the time complexity:

```
int fib(int n) {
    if(n<=1) return n;
    return fib(n-1)+fib(n-2);
}
```

DETAILED EXPLANATION:
The branching sequence mimics standard Fibonacci tree setups, bounding operations to golden ratio tracking models $(1.618)^n$, asymptotically grouped under exponential Big-O criteria.

Time Complexity: $O(2^n)$

Q18. Determine the time complexity:

```
int fact(int n) {
    if(n<=1) return 1;
    return n*fact(n-1);
}
```

DETAILED EXPLANATION:
A linear chain tracking structural sequence where the argument drops incrementally by 1 unit until it reaches the execution baseline.

Time Complexity: O(n)

Q19. Determine the time complexity:

```
void fun(int n) {
    if(n<=0) return;
    fun(n-1);
    cout<<n;
}
```

DETAILED EXPLANATION:
Adding a constant step statement alongside the core sequential recursion pipeline maintains the underlying linear growth rate configuration.

Time Complexity: O(n)

Q20. Determine the time complexity:

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
    cout<<n;
}
```

DETAILED EXPLANATION:
The operation profile relies on bisection, generating the standard base log equation configuration T(n) = T(n/2) + O(1), leading to a clean logarithmic profile.

Time Complexity: O(log n)

Q21. Solve the recurrence relation: T(n) = T(n-1) + 1

DETAILED EXPLANATION:
Unrolling the execution equation reveals a series progression: T(n-k) + k. Evaluating at baseline boundary k = n gives total work proportional directly to n.

Asymptotic Complexity: O(n)

Q22. Solve the recurrence relation: T(n) = T(n-1) + n

DETAILED EXPLANATION:
Complete algebraic expansion gives the natural summation formula up to n elements, which yields the final standard polynomial quadratic factor bounds.

Asymptotic Complexity: $O(n^2)$

Q23. Solve the recurrence relation: T(n) = T(n/2) + 1

DETAILED EXPLANATION:
Master Theorem parameters yield structural equivalence to constant step binary search patterns, proving a logarithmic runtime result.

Asymptotic Complexity: O(log n)

Q24. Solve the recurrence relation: T(n) = 2T(n-1) + 1

DETAILED EXPLANATION:
The multiplicative branch tracking scales values up geometrically at each progression depth step, generating a standard geometric summation bound to exponential curves.

Asymptotic Complexity: $O(2^n)$

Q25. Solve the recurrence relation: T(n) = T(n/2) + n

DETAILED EXPLANATION:
Using Case 3 Master Method checks shows that the linear external cost work element f(n) dominates the log bisection component, pulling the final output constraint directly to a linear profile.

Asymptotic Complexity: O(n)

Q26. Determine the complexity (GATE):

```
for(int i=1;i<=n;i++) {
    for(int j=1;j<=i;j*=2) {}
}
```

DETAILED EXPLANATION:
The nested internal statement steps execute log(i) times per pass. Summing up across all items gives log(1) + log(2) + ... + log(n) = log(n!). Using Stirling's metric simplifies directly to n log n.

Time Complexity: O(n log n)

Q27. Determine the complexity (GATE):

```
for(int i=1;i<=n;i*=2) {
    for(int j=1;j<=n;j++) {}
}
```

DETAILED EXPLANATION:
The outer structure scales logarithmically via geometric doubling while the internal statement remains fully independent, matching standard linear n limits. Their multiplication resolves directly to n log n.

Time Complexity: O(n log n)

Q28. Determine the complexity (GATE):

```
for(int i=n;i>=1;i--) {
    for(int j=i;j>=1;j--) {}
}
```

DETAILED EXPLANATION:
This countdown sequence replicates a classic nested upper triangular parsing pattern, building up an arithmetic progression that simplifies directly to a quadratic polynomial.

Time Complexity: $O(n^2)$

Q29. Determine the complexity (GATE):

```
for(int i=1;i<=n;i++) {
    for(int j=1;j<=n;j*=2) {}
}
```

DETAILED EXPLANATION:
Both boundaries track separate variables independently. The linear envelope loops n times around a standard logarithmic core bisection tracker running log(n) steps, producing an n log n result.

Time Complexity: O(n log n)

Q30. Determine the complexity (GATE):

```
void fun(int n) {
    if(n<=1) return;
    fun(n/2);
    fun(n/2);
}
```

DETAILED EXPLANATION:
This structural layout matches equation patterns tracking T(n) = 2T(n/2) + O(1). Case 1 Master checks establish complete equivalence to a balanced linear parsing profile.

Time Complexity: O(n)